

L2 Implementation - In Simple Words

How the EntityMap was actually built, explained plainly

Layer L2 - Knowledge Graph | PostgreSQL 16 + Apache AGE 1.6.0

Prototype Jun 10 - Prod Jun 15 - Doc due Jun 12 (async review)

1. The big picture

We added a knowledge graph next to the normal crew table, in the SAME database. We used PostgreSQL with an extension called Apache AGE, which lets a normal SQL database also store and query a graph using a language called Cypher. Then we wrote code that reads the 40 crew records and turns them into graph nodes and connections, plus a small web API to ask the graph questions.

2. The pieces we built

File	What it does (plain words)
database/entity_map.py	The brain. Defines the 5 node types and 7 connection types, builds the graph from crew data, and runs the search & traversal queries.
scripts/seed_entity_map.py	A one-command script that fills the graph with the crew data.
api/routes/graph.py	The web endpoints other people/UI call to query the graph.
database/graph_db.py	The only file that talks Cypher to the database (we fixed 3 bugs here).

3. How we turn a crew record into graph pieces

For each crew member we create one Crew node, then connect it to the things it relates to. We always use MERGE ("create only if it isn't already there") so we never get duplicates - MV Pacific Star is one Vessel node even though four crew point at it.

```
for each crew member:
    MERGE a (:Crew) node with their details
    for each certificate -> MERGE (:Certificate) + connect Crew -HOLDS-> it
    connect Crew -CURRENTLY_AT-> (:Port)
    if onboard a ship -> connect Crew -ASSIGNED_TO-> (:Vessel) -CALLS_AT-> (:Port)
    if onboard          -> make a (:Contract) and link Crew -SIGNED-> it
```

4. The questions the graph can answer

a) Search crew by rank + certificate + port

You give any mix of rank, certificate and port; the graph returns the matching crew. Rank is a property on the Crew node; certificate and port are checked by following the HOLDS and CURRENTLY_AT connections.

```
GET /api/v1/graph/crew/search?rank=Chief Officer&certificate=GMDSS&port=Rotterdam
-> 1 result: Piotr Kowalski (took ~5 ms)
```

b) Traverse one crew member's connections

You give a crew id; the graph returns everything connected to them, including two-hop paths like Crew -> Vessel -> Port.

```
GET /api/v1/graph/crew/SOF-2000/traverse
-> CURRENTLY_AT Singapore, ASSIGNED_TO MV Pacific Star, SIGNED CT-SOF-2000 (~30 ms)
```

5. Three real bugs we hit and fixed

These were not obvious, so they are worth recording for whoever touches the graph next:

- > **Colons clash.** Cypher writes connections like [:HOLDS]. The database library thought :HOLDS was a fill-in-the-blank parameter. Fix: send the query as raw text, not through the library's parameter system.
- > **Vertices don't parse as JSON.** AGE tags returned nodes with a ::vertex marker that breaks JSON reading. Fix: always return plain maps of the fields we want, e.g. {name:..., rank:...}.
- > **Writes vanished.** Our create/merge statements weren't being saved because we never told the database to commit. Fix: commit after each query.

6. How we know it works

Check	Result
All 5 node types loaded	Crew 40, Vessel 5, Port 9, Certificate 16, Contract 20
Total graph size	90 nodes, 189 connections
Search by rank+cert+port	Correct single match returned
Multi-hop traversal	Crew -> Vessel -> Port path returned
Speed	5-37 ms (limit was 3000 ms)
No duplicates	5 vessels & 9 ports, not 40 of each

7. How to run it yourself

```
cd backend
echo GRAPH_BACKEND=age >> .env          # turn the graph on
python -m scripts.seed_crew              # load 40 crew rows
python -m scripts.seed_entity_map        # build the graph
```

```
curl localhost:8000/api/v1/graph/summary
```

8. What's next (not built yet)

- > **OpsMap** - add the live workflow state (sign-off -> search -> match -> onboard) as extra connections on the SAME crew nodes.
- > **OrgMap** - add Company and Fleet nodes above the ships, and turn rank into a shared node, to answer 'how many Chief Officers does this fleet need vs have?'.
- > **The golden rule** for both: only ADD connections to existing nodes - never recreate Crew or Vessel - so the graph stays one clean picture with no duplicates.